

Introdução à Análise de Algoritmos

Aula 2

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Roteiro
- 2 Convenções na descrição de algoritmos
- 3 Ordenação por inserção
- 4 Análise de algoritmos
- 5 Projeto de algoritmos
- 6 Exercícios

Visão geral da aula

- ▶ visão geral da estrutura usada para projeto e análise dos algoritmos
- ▶ pseudocódigo usado nos algoritmos
- ▶ ordenação por inserção:
 - ▶ correção
 - ▶ tempo de execução
- ▶ divisão-e-conquista e ordenação por intercalação

Linguagem usada

Pseudocódigo

- ▶ palavras-chave em português
- ▶ similar a C, C++, Java, Python, Pascal
- ▶ clareza e concisão
- ▶ questões de engenharia de software são negligenciadas arbitrariamente

Linguagem usada

Pseudocódigo

- ▶ indentação é usada para indicar blocos de instruções
- ▶ estruturas condicionais: **se**, **se-senão**
- ▶ estruturas de repetição: **enquanto**, **para**, **repita-enquanto**
- ▶ símbolo `//` indica que o restante da linha é um comentário
- ▶ atribuição múltipla: $i = j = e$
- ▶ variáveis são locais aos procedimentos
- ▶ $A[i]$ é usado para indicar o i -ésimo elemento do vetor A ; $A[i..j]$ é usado para indicar um intervalo de $j - i + 1$ elementos do vetor A
- ▶ informações são organizadas em **objetos** compostos de **atributos**: por exemplo, $A.length$

Linguagem usada

Pseudocódigo

- ▶ uma variável representando um vetor ou um objeto é, na verdade, um ponteiro para os dados que representam o vetor ou o objeto
- ▶ parâmetros de um procedimento são sempre passados **por valor**, a menos de vetores e objetos
- ▶ a sentença **devolva** imediatamente termina o procedimento sendo executado e transfere o controle para o ponto onde esse procedimento foi chamado; **devolva** permite devolução de mais que um valor
- ▶ os operadores **E** e **OU** são operadores lógicos
- ▶ sentença **erro** termina a execução do algoritmo sem especificação de como proceder devido a um erro

Problema da ordenação

- ▶ o algoritmo de ordenação por inserção soluciona o problema da ordenação:

PROBLEMA DA ORDENAÇÃO

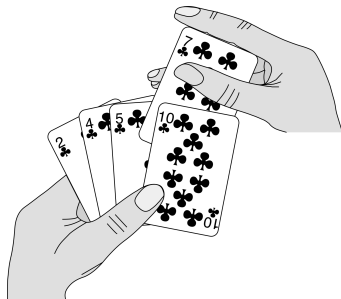
Entrada: uma sequência de n números $\langle a_1, a_2, \dots, a_n \rangle$

Saída: uma permutação $\langle a'_1, a'_2, \dots, a'_n \rangle$ da sequência de entrada de tal forma que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

- ▶ os elementos ou chaves da sequência estão armazenados em um vetor

Ideia do algoritmo

- ▶ o algoritmo de ordenação por inserção funciona da forma como muitas pessoas ordenam uma mão de cartas de baralho



- ▶ usa a **técnica incremental** de projeto de algoritmos: tendo ordenado o vetor $A[1..j-1]$, o elemento $A[j]$ é inserido em seu local correto e obtemos então o vetor $A[1..j]$ ordenado

Algoritmo

INSERTIONSORT(A)

1. **para** $j \leftarrow 2$ **até** $A.length$
2. $chave \leftarrow A[j]$
3. // insere $A[j]$ na sequência ordenada $A[1..j-1]$
4. $i \leftarrow j - 1$
5. **enquanto** $i > 0$ **E** $A[i] > chave$
6. $A[i+1] \leftarrow A[i]$
7. $i \leftarrow i - 1$
8. $A[i+1] \leftarrow chave$

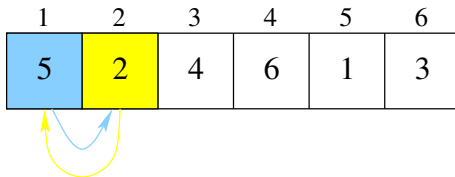
Funcionamento do algoritmo

1	2	3	4	5	6
5	2	4	6	1	3

Funcionamento do algoritmo

1	2	3	4	5	6
5	2	4	6	1	3

Funcionamento do algoritmo



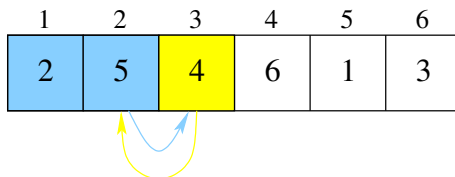
Funcionamento do algoritmo

1	2	3	4	5	6
2	5	4	6	1	3

Funcionamento do algoritmo

1	2	3	4	5	6
2	5	4	6	1	3

Funcionamento do algoritmo



Funcionamento do algoritmo

1	2	3	4	5	6
2	4	5	6	1	3

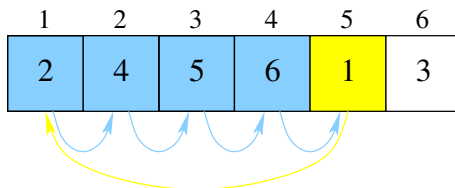
Funcionamento do algoritmo

1	2	3	4	5	6
2	4	5	6	1	3

Funcionamento do algoritmo

1	2	3	4	5	6
2	4	5	6	1	3

Funcionamento do algoritmo



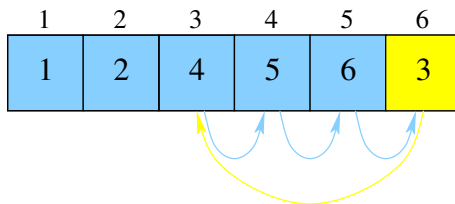
Funcionamento do algoritmo

1	2	3	4	5	6
1	2	4	5	6	3

Funcionamento do algoritmo

1	2	3	4	5	6
1	2	4	5	6	3

Funcionamento do algoritmo



Funcionamento do algoritmo

1	2	3	4	5	6
1	2	3	4	5	6

Correção do algoritmo

Invariantes e a correção do INSERTIONSORT

- ▶ o índice j indica a “carta atual” a ser inserida na mão esquerda
- ▶ no início de cada iteração da estrutura de repetição **para**, o vetor consistindo dos elementos $A[1..j-1]$ constitui a atual mão esquerda com as cartas ordenadas
- ▶ o restante do vetor $A[j+1..n]$ consiste da pilha de cartas ainda na mesa
- ▶ os elementos em $A[1..j-1]$ são aqueles que originalmente estavam armazenados neste intervalo do vetor, mas agora estão ordenados

Correção do algoritmo

Invariantes e a correção do INSERTIONSORT

- ▶ esta propriedade do vetor $A[1..j-1]$ pode ser descrita como um **invariante**:
 - ▶ no começo de cada iteração da estrutura de repetição **para** das linhas 1–8, o vetor $A[1..j-1]$ consiste dos elementos originalmente em $A[1..j-1]$, mas em ordem crescente
- ▶ usamos invariantes para ajudar a compreender por que um algoritmo está correto

Correção do algoritmo

- ▶ devemos mostrar três propriedades sobre um invariante:
 - ▶ **Inicialização:** é verdadeiro antes da primeira iteração da estrutura de repetição;
 - ▶ **Manutenção:** se é verdadeiro antes de uma iteração da estrutura de repetição, permanece verdadeiro antes da próxima iteração;
 - ▶ **Término:** quando a estrutura de repetição termina, o invariante nos dá uma propriedade útil que nos permite mostrar que o algoritmo está correto
- ▶ similar à indução matemática

Correção do algoritmo

INSERTIONSORT(A)

1. **para** $j \leftarrow 2$ **até** $A.length$
2. $chave \leftarrow A[j]$
3. // insere $A[j]$ na sequência ordenada $A[1..j-1]$
4. $i \leftarrow j - 1$
5. **enquanto** $i > 0$ **E** $A[i] > chave$
6. $A[i+1] \leftarrow A[i]$
7. $i \leftarrow i - 1$
8. $A[i+1] \leftarrow chave$

Correção do algoritmo

Invariantes e a correção do INSERTIONSORT

- **Inicialização:** antes da primeira iteração da estrutura de repetição **para**, temos que $j = 2$; dessa forma, o vetor $A[1..j-1]$ consiste de um único elemento $A[1]$, que é de fato o elemento original em $A[1]$; além disso, $A[1]$ está ordenado, o que mostra que o invariante é verdadeiro antes da primeira iteração;

Correção do algoritmo

Invariantes e a correção do INSERTIONSORT

- **Manutenção:** o corpo de instruções da estrutura de repetição **para** trabalha para movimentar os elementos $A[j-1], A[j-2], \dots$ uma posição para direita, até encontrar a posição correta para $A[j]$ (linhas 4–7), momento em que insere no local correto do vetor A o valor de $A[j]$ (linha 8); o vetor $A[1..j]$ consiste então dos elementos originalmente em $A[1..j]$, mas rearranjados em ordem crescente; incrementar j para a próxima iteração da estrutura de repetição **para** preserva assim o invariante;

Correção do algoritmo

Invariantes e a correção do INSERTIONSORT

- **Término:** a condição de parada da estrutura de repetição **para** é $j > A.length = n$; como cada iteração incrementa j em uma unidade, temos que $j = n + 1$; substituindo $n + 1$ no lugar de j no invariante, temos que o vetor $A[1..n]$ consiste dos elementos originalmente em $A[1..n]$, mas rearranjados em ordem crescente; o vetor $A[1..n]$ é o vetor completo, isto é, o vetor de entrada está rearranjado em ordem crescente; portanto, o algoritmo está correto

Generalidades

- ▶ **analisar** um algoritmo significa prever os recursos que o algoritmo requer
- ▶ em geral, queremos medir o tempo computacional de um algoritmo, mas eventualmente também podemos nos preocupar com memória, largura de banda de comunicação, etc
- ▶ antes de analisar um algoritmo, devemos ter um modelo da tecnologia em que o mesmo será implementado, com seus custos associados
- ▶ fixaremos um modelo de implementação genérico de um processador, chamado **máquina de acesso aleatório** (RAM), onde os algoritmos serão transformados em programas
- ▶ cada instrução (aritmética, de movimentação de dados e de controle) no modelo RAM tem custo constante
- ▶ os tipos de dados são números inteiros e números de ponto flutuante, com limite no número de bytes de armazenamento

Generalidades

- ▶ analisar um algoritmo no modelo RAM pode ser um desafio
- ▶ ferramentas matemáticas necessárias podem incluir combinatória, teoria das probabilidades, destreza algébrica e habilidade de identificar os termos mais significativos em fórmulas
- ▶ como o comportamento de um algoritmo pode ser diferente para cada entrada possível, precisamos de uma maneira de resumir seu comportamento em uma fórmula simples e compreensível

Análise do INSERTIONSORT

- ▶ o tempo gasto pelo algoritmo INSERTIONSORT depende da entrada: ordenar mil números é mais demorado que ordenar três números
- ▶ além disso, o algoritmo pode gastar diferentes quantidades de tempo para ordenar duas sequências de entrada de mesmo tamanho, dependendo de quão ordenadas elas já estão
- ▶ em geral, o tempo gasto por um algoritmo cresce com o tamanho de sua entrada e, por isso, é usual descrever o tempo de execução de um programa como uma função do tamanho de sua entrada

Análise do INSERTIONSORT

- ▶ **tamanho da entrada** depende do problema: número de itens de entrada ou número total de bits necessários para representar a entrada; às vezes, mais de um número representa a entrada
- ▶ **tempo de execução** de um algoritmo sobre uma entrada particular é o número de operações primitivas ou passos executados
- ▶ uma quantidade de tempo constante é necessária para executar cada linha de um algoritmo
- ▶ isto é, a execução da i -ésima linha do algoritmo gasta tempo c_i , onde c_i é uma constante

Análise do INSERTIONSORT

INSERTIONSORT(A)		custo	vezes
1.	para $j \leftarrow 2$ até $A.length$	c_1	n
2.	chave $\leftarrow A[j]$	c_2	$n - 1$
3.	// insere $A[j]$ na sequência ordenada $A[1..j - 1]$	0	$n - 1$
4.	$i \leftarrow j - 1$	c_4	$n - 1$
5.	enquanto $i > 0$ E $A[i] > \text{chave}$	c_5	$\sum_{j=2}^n t_j$
6.	$A[i + 1] \leftarrow A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7.	$i \leftarrow i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8.	$A[i + 1] \leftarrow \text{chave}$	c_8	$n - 1$

Análise do INSERTIONSORT

- ▶ o tempo de execução do INSERTIONSORT é então a soma dos tempos de execução de cada sentença executada:

$$\begin{aligned}
 T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).
 \end{aligned}$$

Análise do INSERTIONSORT

- ▶ mesmo para entradas de um dado tamanho, o tempo de execução de um algoritmo pode depender de *quais* entradas deste tamanho são fornecidas
- ▶ no INSERTIONSORT, o melhor caso ocorre quando o vetor de entrada é fornecido em ordem crescente
- ▶ neste caso, para cada $j = 2, 3, \dots, n$, temos que $A[i] \leq \text{chave}$ na linha 5, quando i foi inicializado com $j - 1$
- ▶ assim, $t_j = 1$ para todo $j = 2, 3, \dots, n$ e o tempo de execução do melhor caso do INSERTIONSORT é:

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

- ▶ podemos expressar este tempo de execução como $an + b$, para constantes a e b que dependem exclusivamente dos custos das instruções c_i ; ou seja, uma **função linear de n**

Análise do INSERTIONSORT

- ▶ o pior caso para o INSERTIONSORT ocorre quando o vetor de entrada é fornecido em ordem decrescente
- ▶ neste caso, todo elemento $A[j]$ deve ser comparado com cada elemento do vetor ordenado $A[1..j-1]$
- ▶ assim, $t_j = j$ para todo $j = 2, 3, \dots, n$

Análise do INSERTIONSORT

- assim, no pior caso, o tempo de execução do INSERTIONSORT é:

$$\begin{aligned}
 T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n j \\
 &\quad + c_6 \sum_{j=2}^n (j-1) + c_7 \sum_{j=2}^n (j-1) + c_8(n-1) \\
 &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\
 &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\
 &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 - \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\
 &\quad - (c_2 + c_4 + c_5 + c_8)
 \end{aligned}$$

Análise do INSERTIONSORT

- ▶ podemos expressar este tempo de execução de pior caso como $an^2 + bn + c$ para constantes a, b e c que dependem dos custos das instruções c_i
- ▶ uma função quadrática de n

Casos na análise de algoritmos

Melhor caso, pior caso ou caso médio?

- ▶ usualmente nos concentramos apenas no **tempo de execução de pior caso**
- ▶ é o maior tempo gasto para *qualquer* entrada de tamanho n , isto é, fornece um limitante superior do tempo de execução para qualquer entrada de tamanho n
- ▶ o pior caso ocorre muito frequentemente para muitos algoritmos
- ▶ o “caso médio” é frequentemente tão ruim quanto o pior caso

Caso médio

Análise de caso médio

- ▶ **análise probabilística**
- ▶ o escopo da análise de caso médio é limitado, porque muitas vezes não sabemos o que significa a entrada “média” para um problema particular
- ▶ consideramos então que todas as entradas de um determinado tamanho são igualmente prováveis
- ▶ na prática esta condição pode ser violada, mas em alguns casos podemos usar um **algoritmo aleatorizado**, que faz escolhas aleatórias, para permitir uma análise probabilística que fornece um tempo de execução **esperado**

Funções que representam tempo de execução

Ordem de crescimento

- ▶ usamos algumas abstrações simplificadas para facilitar a análise de um algoritmo
- ▶ por exemplo, ignoramos o custo real de cada instrução e usamos constantes c_i para representar esses custos
- ▶ além dos custos reais de cada instrução, acabamos por ignorar também os custos abstratos c_i : expressamos o tempo de execução de pior caso do INSERTIONSORT como $an^2 + bn + c$, para constantes a, b e c que dependem dos custos das instruções c_i
- ▶ a **taxa de crescimento** ou **ordem de crescimento** do tempo de execução é o que realmente nos interessa, o que nos leva a fazer mais uma simplificação

Funções que representam tempo de execução

Ordem de crescimento

- ▶ consideramos apenas o termo dominante de uma fórmula, por exemplo an^2 , já que os outros termos menores são praticamente insignificantes para grandes valores de n
- ▶ também ignoramos o coeficiente constante do termo dominante, já que fatores constantes são menos significativos que a taxa de crescimento na determinação da eficiência computacional para grandes entradas
- ▶ no INSERTIONSORT, quando ignoramos termos de menor ordem e o coeficiente constante do termo dominante, sobra o termo n^2
- ▶ dizemos então que o INSERTIONSORT tem tempo de execução de pior caso $\Theta(n^2)$
- ▶ consideramos um algoritmo mais eficiente que outros se seu tempo de execução de pior caso tem a menor taxa de crescimento

Divisão-e-conquista

- ▶ uma alternativa à técnica incremental usada na ordenação por inserção é chamada **divisão-e-conquista**
- ▶ esta técnica permite construir algoritmos de ordenação cujo tempo de execução de pior caso é muito menor que do INSERTIONSORT
- ▶ os tempos de execução dos algoritmos de divisão-e-conquista são facilmente determinados

Divisão-e-conquista

- ▶ muitos problemas têm **estrutura recursiva**: para solucionar o problema, chamam a si próprios uma ou mais vezes sobre subproblemas relacionados
- ▶ estes algoritmos em geral seguem a técnica de **divisão-e-conquista**: dividem o problema em vários subproblemas que são similares ao problema original, mas são menores em tamanho, solucionam esses subproblemas e então combinam essas soluções produzindo uma solução do problema original

Divisão-e-conquista

A técnica de divisão-e-conquista envolve três passos em cada nível de recursão:

Divida o problema em um número de subproblemas que são instâncias menores do mesmo problema

Conquiste os subproblemas, solucionando-os recursivamente; porém, se os tamanhos dos subproblemas são pequenos o suficiente, solucione os subproblemas de maneira imediata

Combine as soluções dos subproblemas na solução do problema original

Divisão-e-conquista

MERGESORT

- Divida** a sequência de n elementos a ser ordenada em duas subsequências de $n/2$ elementos cada
- Conquiste** ordene as duas subsequências recursivamente usando o MERGESORT
- Combine** ou intercale as duas subsequências ordenadas para produzir o vetor resposta ordenado

Divisão-e-conquista

- ▶ a recursão chega ao fim quando a sequência a ser ordenada tem tamanho 1, caso em que não há trabalho a fazer
- ▶ a operação chave do algoritmo MERGESORT é a intercalação de duas sequências ordenadas no passo “combinar”
- ▶ a intercalação é um procedimento auxiliar $\text{MERGE}(A, p, q, r)$, onde A é um vetor e p, q e r são índices do vetor tais que $p \leq q < r$
- ▶ os vetores $A[p..q]$ e $A[q + 1..r]$ estão ordenados
- ▶ o procedimento intercala esses vetores em um único vetor ordenado que substitui o atual vetor $A[p..r]$

Intercalação

```

MERGE( $A, p, q, r$ )
01.   $n_1 \leftarrow q - p + 1$ 
02.   $n_2 \leftarrow r - q$ 
03.  sejam  $L[1..n_1 + 1]$  e  $R[1..n_2 + 1]$  novos vetores
04.  para  $i \leftarrow 1$  até  $n_1$ 
05.       $L[i] \leftarrow A[p + i - 1]$ 
06.  para  $j \leftarrow 1$  até  $n_2$ 
07.       $R[j] \leftarrow A[q + j]$ 
08.   $L[n_1 + 1] \leftarrow \infty$ 
09.   $R[n_2 + 1] \leftarrow \infty$ 
10.   $i \leftarrow 1$ 
11.   $j \leftarrow 1$ 
12.  para  $k \leftarrow p$  até  $r$ 
13.      se  $L[i] \leq R[j]$ 
14.           $A[k] \leftarrow L[i]$ 
15.           $i \leftarrow i + 1$ 
16.      senão
17.           $A[k] \leftarrow R[j]$ 
18.           $j \leftarrow j + 1$ 

```

Intercalação

MERGE

- ▶ linha 1 computa o tamanho n_1 do vetor $A[p..q]$
- ▶ linha 2 computa o tamanho n_2 do vetor $A[q+1..r]$
- ▶ vetores L e R , de tamanhos $n_1 + 1$ e $n_2 + 1$, respectivamente, são criados na linha 3
- ▶ a estrutura de repetição **para** nas linhas 4–5 copia o vetor $A[p..q]$ em $L[1..n_1]$
- ▶ a estrutura de repetição **para** nas linhas 6–7 copia o vetor $A[q+1..r]$ em $R[1..n_2]$
- ▶ linhas 8–9 adicionam sentinelas no final dos vetores L e R

Intercalação

MERGE

- ▶ linhas 10–18 executam $r - p + 1$ passos, mantendo o seguinte invariante:

no começo de cada iteração da estrutura de repetição **para** das linhas 12–18, o vetor $A[p..k-1]$ contém os $k - p$ menores elementos de $L[1..n_1 + 1]$ e $R[1..n_2 + 1]$, ordenados; além disso, $L[i]$ e $R[j]$ são os menores elementos de seus vetores que não foram ainda copiados de volta para A

Intercalação

Correção do MERGE

- **Inicialização:** antes da primeira iteração do laço, temos que $k = p$ e assim o vetor $A[p..k-1]$ é vazio; este vetor vazio contém os $k - p = 0$ menores elementos de L e R e, como $i = j = 1$, ambos $L[i]$ e $R[j]$ são os menores elementos de seus vetores que ainda não foram copiados de volta para A ;

Intercalação

Correção do MERGE

- **Manutenção:** suponha primeiro que $L[i] \leq R[j]$; então $L[i]$ é o menor elemento ainda não copiado de volta em A ; como $A[p..k-1]$ contém os $k-p$ menores elementos, depois que a linha 14 copia $L[i]$ em $A[k]$, o vetor $A[p..k]$ conterá os $k-p+1$ menores elementos; com o incremento de k e i , o invariante é reestabelecido para a próxima iteração; se, ao contrário $L[i] > R[j]$, então as linhas 16–18 executam os passos apropriados para manter o invariante;

Intercalação

Correção do MERGE

- **Término:** ao final, $k = r + 1$; pelo invariante, o vetor $A[p..k - 1]$, que é o vetor $A[p..r]$, contém os $k - p = r - p + 1$ menores elementos de $L[1..n_1]$ e $R[1..n_2]$, ordenados; os vetores L e R juntos têm $n_1 + n_2 + 2 = r - p + 3$ elementos; todos, a menos de 2, elementos foram copiados de volta para A e esses dois maiores elementos são os sentinelas

Intercalação

Tempo de execução do MERGE

- ▶ faça $n = r - p + 1$
- ▶ as linhas 1–3 e 8–11 gastam tempo constante
- ▶ as duas estruturas de repetição **para** das linhas 4–7 gastam tempo $\Theta(n_1 + n_2) = \Theta(n)$
- ▶ a estrutura de repetição **para** das linhas 12–18 é executada n vezes, onde cada execução gasta tempo constante
- ▶ portanto, o tempo de execução do MERGE é $\Theta(n)$

Ordenação por intercalação

MERGESORT(A, p, r)

01. **se** $p < r$

02. $q \leftarrow \lfloor (p + r) / 2 \rfloor$

03. MERGESORT(A, p, q)

04. MERGESORT($A, q + 1, r$)

05. MERGE(A, p, q, r)

Análise de algoritmos de divisão-e-conquista

- ▶ quando um algoritmo tem uma ou mais chamadas recursivas, é usual descrever seu tempo de execução por uma **equação de recorrência** ou **recorrência**
- ▶ uma recorrência descreve o tempo de execução de um problema com entrada de tamanho n em termos do tempo de execução do mesmo problema sobre entradas de tamanhos menores
- ▶ podemos usar ferramentas matemáticas para solucionar recorrências e apresentar limitantes sobre o desempenho do algoritmo
- ▶ uma recorrência para o tempo de execução de um algoritmo de divisão-e-conquista tem as características dos três passos dessa técnica

Análise de algoritmos de divisão-e-conquista

- ▶ seja $T(n)$ o tempo de execução de um algoritmo sobre um problema de tamanho n
- ▶ se o tamanho do problema é suficientemente pequeno, digamos $n \leq c$ para alguma constante c , então uma solução simples pode ser obtida em tempo constante $\Theta(1)$
- ▶ suponha que a divisão do problema forneça a subproblemas, cada subproblema tendo $1/b$ do tamanho do problema original
- ▶ o tempo gasto para solucionar um subproblema de tamanho n/b é $T(n/b)$ e, assim, o tempo gasto para solucionar a subproblemas é $aT(n/b)$
- ▶ se gastamos $D(n)$ para dividir o problema em subproblemas e $C(n)$ para combinar as soluções do subproblemas na solução do problema original, temos a recorrência

$$T(n) = \begin{cases} \Theta(1), & \text{se } n \leq c, \\ aT(n/b) + D(n) + C(n), & \text{caso contrário.} \end{cases}$$

Análise do MERGESORT

MERGESORT

Divida: este passo apenas calcula o “meio” do vetor, o que gasta tempo constante, isto é, $\Theta(1)$

Conquiste: recursivamente, dois subproblemas de tamanho $n/2$ são solucionados, o que contribui com $2T(n/2)$ para o tempo de execução

Combine o tempo de execução do MERGE sobre um vetor de n elementos é $\Theta(n)$ e assim $C(n) = \Theta(n)$

Análise do MERGESORT

- ▶ dessa forma, a recorrência para o tempo de execução de pior caso $T(n)$ do MERGESORT é dada por

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 2T(n/2) + \Theta(1) + \Theta(n), & \text{se } n > 1. \end{cases}$$

Análise do MERGESORT

- ▶ dessa forma, a recorrência para o tempo de execução de pior caso $T(n)$ do MERGESORT é dada por

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 2T(n/2) + \Theta(n), & \text{se } n > 1. \end{cases}$$

Análise do MERGESORT

- ▶ dessa forma, a recorrência para o tempo de execução de pior caso $T(n)$ do MERGESORT é dada por

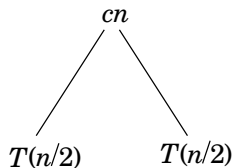
$$T(n) = \begin{cases} a, & \text{se } n = 1, \\ 2T(n/2) + cn, & \text{se } n > 1, \end{cases}$$

onde a constante a representa o tempo necessário para resolver problemas de tamanho 1, bem como o tempo gasto nos processos de dividir e conquistar

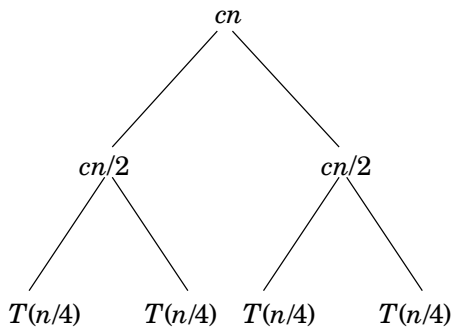
Análise do MERGESORT

$$T(n)$$

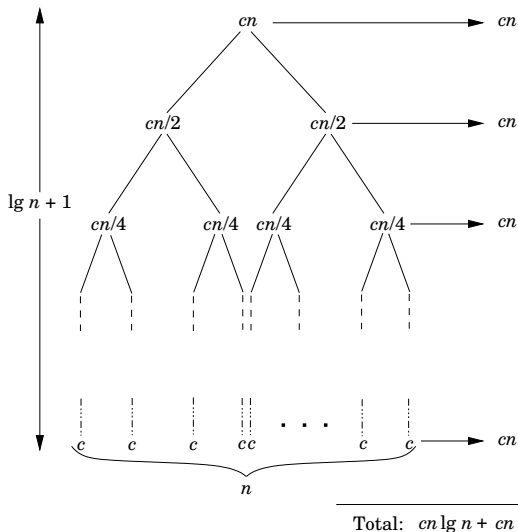
Análise do MERGESORT



Análise do MERGESORT



Análise do MERGESORT



Análise do MERGESORT

- ▶ podemos usar o “teorema mestre” ou uma árvore de recursão para mostrar que $T(n) = \Theta(n \lg n)$, onde $\lg = \log_2$
- ▶ como a função logarítmica cresce mais lentamente que a função linear, para entradas suficientemente grandes, o MERGESORT com seu tempo de execução $\Theta(n \lg n)$ supera o INSERTIONSORT, cujo tempo de execução é $\Theta(n^2)$ no pior caso

Exercícios

- 2.1-1 Ilustre o funcionamento do INSERTIONSORT sobre o vetor $A = \langle 31, 41, 59, 26, 41, 58 \rangle$
- 2.1-2 Reescreva o algoritmo INSERTIONSORT para rearranjar uma sequência de entrada em ordem decrescente
- 2.1-3 Considere o seguinte problema:

PROBLEMA DA BUSCA

Entrada: uma sequência de n números $A = \langle a_1, a_2, \dots, a_n \rangle$ e um valor v
Saída: um índice i tal que $v = A[i]$ ou um valor especial $k \notin \{1, \dots, n\}$ indicando que v não ocorre em A

Escreva um pseudocódigo para a **busca linear**, que percorre a sequência em busca de v . Usando um invariante, prove que o algoritmo está correto.

Exercícios

- 2.2-1 Expresse a função $n^3/1000 - 100n^2 - 100n + 3$ nos termos da notação Θ
- 2.2-2 Considere a ordenação de n números armazenados em um vetor A encontrando primeiro o menor elemento de A e trocando-o com o elemento $A[1]$. Então encontre o segundo menor elemento de A e troque-o com $A[2]$. Continue desta maneira para os primeiros $n - 1$ elementos de A . Escreva um pseudocódigo para este algoritmo, que é conhecido como **ordenação por seleção**. Qual invariante o algoritmo mantém? Por que o algoritmo precisa ser executado somente para os primeiros $n - 1$ elementos, ao invés de para todos os n elementos? Forneça o tempo de execução do melhor caso e do pior caso do algoritmo usando a notação Θ

Exercícios

2.2-3 Considere novamente a busca linear [veja o exercício 2.1-3]. Quantos elementos da sequência de entrada precisam ser verificados na média, considerando que o elemento procurado é igualmente provável a qualquer outro no vetor? E no pior caso? Quais são os tempos de execução de caso médio e de pior caso em termos da notação Θ ? Justifique suas respostas

Exercícios

2.3-1 Ilustre a operação do MERGESORT sobre o vetor

$A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$

2.3-3 Use indução matemática para mostrar que quando n é uma potência de 2, a solução da recorrência

$$T(n) = \begin{cases} 2, & \text{se } n = 2, \\ 2T(n/2) + n, & \text{se } n = 2^k, \text{ para } k > 1, \end{cases}$$

é $T(n) = n \lg n$

2.3-4 Podemos escrever um procedimento recursivo para a ordenação por inserção, como segue. Para ordenar o vetor $A[1..n]$, ordenamos recursivamente $A[1..n-1]$ e então inserimos $A[n]$ no vetor ordenado $A[1..n-1]$. Escreva uma recorrência para o tempo de execução desta versão recursiva da ordenação por inserção

Exercícios

- 2.3-5 Considerando novamente o problema da busca, observe que se a sequência de entrada A é ordenada, podemos comparar o elemento do meio da sequência com v e eliminar metade da sequência. O algoritmo de **busca binária** repete este processo, dividindo ao meio o tamanho da porção restante da sequência a cada passo. Escreva um pseudocódigo para a busca binária. Argumente que o tempo de execução de pior caso da busca binária é $\Theta(\lg n)$
- 2.3-6 Observe que a estrutura de repetição **enquanto** das linhas 5–7 do algoritmo INSERTIONSORT usa busca linear no vetor ordenado $A[1..j-1]$. Será que podemos usar a busca binária para melhorar o tempo de execução de pior caso da ordenação por inserção para $\Theta(n \lg n)$?

Exercícios

2.3-7 Escreva um algoritmo com tempo de execução $\Theta(n \lg n)$ que, dado um conjunto S de n números inteiros e um outro número inteiro x , determina se existem dois números em S cuja soma é exatamente x

Problemas

2-2 Correção do BUBBLESORT

O algoritmo de ordenação por trocas sucessivas, mais conhecido como BUBBLESORT, é muito popular, mas ineficiente.

```
BUBBLESORT(A)  
1.  para  $i \leftarrow 1$  até  $A.length - 1$   
2.      para  $j \leftarrow A.length$  até  $i + 1$  passo  $-1$   
3.          se  $A[j] < A[j - 1]$   
4.              troque  $A[j]$  com  $A[j - 1]$ 
```

Problemas

- (a) Denote por A' a saída do `BUBBLESORT`(A). Para provar que `BUBBLESORT` está correto, devemos provar que ele termina e que

$$A'[1] \leq A'[2] \leq \dots \leq A'[n]$$

onde $n = A.length$. Para mostrar que o `BUBBLESORT` de fato ordena os elementos de entrada, o que mais é necessário provar?

- (b) Descreva um invariante das linhas 1–4 do algoritmo e prove que ele é verdadeiro.
- (c) Qual o tempo de execução de pior caso do `BUBBLESORT`? Compare este tempo com o tempo de execução do `INSERTIONSORT`.

Problemas

2-4 Inversões

Seja $A[1..n]$ um vetor de n números inteiros distintos. Se $i < j$ e $A[i] > A[j]$ então o par (i, j) é chamado uma **inversão** de A .

- (a) Liste as cinco inversões do vetor $\langle 2, 3, 8, 6, 1 \rangle$.
- (b) Qual vetor com elementos do conjunto $\{1, 2, \dots, n\}$ tem o maior número de inversões? Quantas são?
- (c) Qual a relação entre o tempo de execução da ordenação por inserção e o número de inversões em um vetor de entrada? Justifique sua resposta.
- (d) Modificando a ordenação por intercalação, escreva uma função eficiente, com tempo de execução $\Theta(n \lg n)$, que determine o número de inversões em uma permutação de n elementos.